

Landmålerberegner

Udviklingsforløbet — fra tomt projekt til færdig app

Dette dokument fortæller historien om, hvordan Landmålerberegneren blev bygget, trin for trin — inklusive de omveje, fejl og opdagelser undervejs. Formålet er ikke kun at vise det færdige resultat, men at vise selve *processen*: hvordan man fejlsøger, tager beslutninger, og nogle gange må rulle noget tilbage igen. Det er en naturlig del af at bygge software.

Indhold

Fase 1 — Grundlæggende opsætning

Fase 2 — Kortet og de første målepunkter

Fase 3 — Areal, omkreds og redigering

Fase 4 — Flere kortlag

Fase 5 — Koordinatsystemer

Fase 6 — Dansk sprog

Fase 7 — Manuel koordinatindtastning

Fase 8 — Skelpunkter fra Dataforsyningen (den lange rejse)

Fase 9 — Skellinjer

Fase 10 — Ortofoto som baggrundskort

Fase 11 — Forsøget med matrikelnumre (og hvorfor det blev rullet tilbage)

Fase 12 — Oprydning: omstrukturering af koden

Fase 13 — Delte knapper og adressesøgning

Fase 14 — Git og GitHub

Opsummering — hvad endte vi med?

Fase 1 — Grundlæggende opsætning

Vi startede med at bekræfte, at Node.js, npm og Git var installeret, og brugte derefter Vite til at oprette et tomt React-projekt: `npm create vite@latest surveyor-app -- --template react`. Derefter ryddede vi Vites standard-indhold (logoer, tæller-knap) væk, så vi havde en helt blank side at bygge videre på. Git blev initialiseret med det samme, så vi havde et sikkerhedsnet fra starten.

■ **Læring:** Start altid med et rent, testet fundament, og commit det, før du bygger videre — så har du altid et sted at vende tilbage til.

Fase 2 — Kortet og de første målepunkter

Første version brugte almindelige talfelter til X/Y-koordinater. Men et rigtigt landmålværktøj bør vise punkterne visuelt — så vi skiftede hurtigt til et interaktivt kort med Leaflet og react-leaflet. Klik på kortet tilføjede et punkt; en lille hjælpekomponent (ClickHandler) lyttede efter klik-hændelser og rapporterede dem videre til appens data.

■■ Leaflets standard-markører virker ikke automatisk i Vite-projekter — det kræver en lille rettelse, der manuelt sætter ikonernes webadresser.

Fase 3 — Areal, omkreds og redigering

Med punkter på kortet kunne vi beregne det faktiske areal og omkreds af den flade, de dannede — med den klassiske "shoelace"-formel til areal, og simpel afstandsberegning til omkreds. Da lat/long er gradmål (ikke meter), omregnede vi først punkterne til lokale meterkoordinater med en flad tilnærmelse, før beregningen.

Derefter gjorde vi punkterne redigerbare direkte på kortet: **træk** for at flytte et punkt, **højreklik** for at fjerne det seneste. Det gjorde værktøjet langt mere brugbart end kun at kunne tilføje/slette ét ad gangen fra en liste.

Fase 4 — Flere kortlag

Vi tilføjede en lag-vælger (Leaflets LayersControl) med tre baggrundskort: almindelige gader (OpenStreetMap), satellitbilleder (Esri), og terræn/højdekurver (OpenTopoMap) — alle gratis, offentlige tjenester uden krav om nøgle.

Fase 5 — Koordinatsystemer

En rigtig landmåler arbejder ikke kun i GPS-koordinater (WGS84) — derfor tilføjede vi biblioteket proj4 og lod brugeren skifte mellem WGS84, UTM Zone 32N, og senere DKTM3 og System 34 Sjælland. Både kortets punkttabel og et nyt felt til manuel indtastning fulgte automatisk det valgte system.

■■ System 34 Sjælland viste sig at være særligt: den officielle, præcise konvertering kræver en specifik dansk "grid-shift"-fil, som vores browser-bibliotek ikke understøtter. Vi valgte at implementere en tilnærmet version med en tydelig advarsel i stedet for at undlade funktionen helt.

Fase 6 — Dansk sprog

Al tekst i brugergrænsefladen blev oversat til dansk — overskrifter, knapper, tabelheadere, kortlagenes navne og fejlbeskeder. Kode og kommentarer forblev på engelsk/teknisk, da det kun er brugerfladen, der skal være letforståelig for slutbrugeren.

Fase 7 — Manuel koordinatindtastning

Ikke alle punkter kommer fra et klik på kortet — nogle gange har man allerede koordinater fra en anden kilde. Vi tilføjede et inputfelt, der automatisk viser de rigtige labels (Breddegrad/Længdegrad, eller Øst/Nord i meter) alt efter det valgte koordinatsystem, og som omregner input til WGS84 internt (da kortet altid arbejder i WGS84), før punktet tilføjes.

Fase 8 — Skelpunkter fra Dataforsyningen (den lange rejse)

Dette blev den mest lærerige — og mest udfordrende — del af hele projektet. Målet var simpelt: vise rigtige matrikulære skelpunkter fra Dataforsyningen på kortet. Vejen derhen var alt andet end simpel.

Trin 1: Det forkerte system

Vi opdagede, at "Dataforsyningen" faktisk dækker over **to forskellige platforme** med hver sit login: det ældre dataforsyningen.dk (simpel token-baseret adgang) og det nyere datafordeler.dk (mere avanceret, med API-nøgler og OAuth). Første forsøg byggede på den forkerte platforms dokumentation.

Trin 2: WFS-tjenesten der ikke længere fandtes

Vi forsøgte at bygge kald mod en klassisk WFS-tjeneste (en almindelig standard til geodata), men fik konsekvent 401-fejl. Efter grundig fejlsøgning (forkert store/små bogstaver i parameternavne, forkert nøgletype osv.) fandt vi den egentlige årsag: Matriklen var netop blevet moderniseret, og den klassiske WFS-tjeneste fandtes ikke længere i den form — kun Fildownload og en ny **GraphQL**-baseret API.

■ **Læring:** Når intet giver mening efter flere fejlsøgningsrunder, så tjek om selve grundlaget (platformen/tjenesten) er det rigtige, før du bliver ved med at finpudse detaljerne.

Trin 3: At finde det rigtige GraphQL-endpoint

GraphQL-API'et krævede det korrekte "register" og "version" i selve webadressen (fx MAT/v2). Det fandt vi ved systematisk at afprøve versionsnumre, indtil et virkede — bekræftet ved, at et gyldigt kald returnerede en skema-fil, vi kunne læse direkte.

Trin 4: At læse skemaet manuelt

Uden en automatisk "query builder" måtte vi selv finde de rigtige felter ved at søge i den downloadede skema-fil: type MAT_Skelpunkt, derefter dens geometri-type, derefter filter-typerne, indtil vi havde nok information til at skrive en korrekt, geografisk filtreret forespørgsel.

Trin 5: En sikker proxy for nøglen

For ikke at afsløre API-nøglen i browseren byggede vi en lille proxy ind i Vite (`vite.config.js`), som modtager kald fra browseren på en lokal adresse, tilføjer nøglen usynligt på serversiden, og videresender kaldet til Dataforsyningen.

```
'/api/skelpunkter': {
  target: 'https://graphql.datafordeler.dk',
  rewrite: () => `/MAT/v2?apiKey=${DATAFORDELER_APIKEY}`
}
```

Trin 6: Flere mindre forhindringer

- Forespørgsler med flere "rod-felter" blev afvist — løsning: to separate parallelle kald i stedet for ét kombineret.
- Fejlbeskeder blev først skjult af vores egen generiske fejlhåndtering — rettet til at vise serverens rigtige fejltekst.
- Data er "bitemporal" (har både registrerings- og virkningstidspunkter) — kræver et tidspunktsfilter i hver forespørgsel.

■ **Læring:** Byg fejlhåndtering, der viser dig den fulde, rigtige fejlbesked fra serveren — en generisk "noget gik galt" gør fejlsøgning meget langsommere.

Fase 9 — Skellinjer

Med skelpunkter på plads var skellinjer (Matrikelskel) et naturligt næste skridt — en anden entitet i samme GraphQL-skema, denne gang med en linje-geometri (WKT-format "LINESTRING") i stedet for et enkelt punkt. Vi brugte "intersects" som geografisk filter i stedet for "within", så linjer der krydser kortudsnittets kant også blev vist, ikke kun linjer der ligger 100% inden i.

Fase 10 — Ortofoto som baggrundskort

Vi tilføjede Dataforsyningens 10 cm-luftfoto som et ekstra kortlag. Her stødte vi på endnu en detalje: den almindelige udgave af ortofoto-tjenesten bruger dansk UTM32-projektion, som Leaflet ikke forstår direkte (Leaflet er bygget til "Web Mercator"). Løsningen var at finde og bruge den specifikke "Web Mercator"-udgave af tjenesten, som findes netop til brug i webkort som vores.

Fase 11 — Forsøget med matrikelnumre (og hvorfor det blev rullet tilbage)

Vi forsøgte at vise selve matrikelfladerne (jordstykkerne) med deres matrikelnummer skrevet inde i arealet. Det krævede at hente to sammenkædede entiteter (JordstykkeTemaflade for geometrien, Jordstykke for selve nummeret) og en langt mere avanceret WKT-parser, der korrekt kunne håndtere flerdelte flader og huller i polygoner.

Efter grundig fejlsøgning (forkerte antagelser om dubletter, forkert WKT-parsing af huller, manglende POLYGON- vs. MULTIPOLYGON-håndtering) opdagede vi den egentlige årsag: Den nye GraphQL-API havde reelle **datahuller** — nogle jordstykker fandtes slet ikke i denne specifikke entitet endnu, selvom

de tydeligvis fandtes i Dataforsyningens egen, ældre, officielle kortviser. Det blev bekræftet ved direkte at sammenligne med den officielle kortviser.

Da problemet lå i selve datakilden — ikke i vores kode — og ikke kunne rettes fra vores side, valgte vi at fjerne funktionen igen, fremfor at levere en upålidelig oplevelse.

■ **Læring:** Ikke alle problemer kan løses med bedre kode. Når en grundig, systematisk fejlsøgning peger på en ægte begrænsning i datakilden, er det ofte den rigtige beslutning at fjerne funktionen igen og vente på bedre data, fremfor at aflevere noget upålideligt.

Fase 12 — Oprydning: omstrukturering af koden

På dette tidspunkt var App.jsx vokset til over 500 linjer og blevet svær at overskue. Vi delte filen op efter ansvar: ren logik (koordinatberegning, API-kald) flyttede til en utils/-mappe, mens hver visuel del af skærmen blev sin egen komponent i en components/-mappe. Alt fungerede identisk bagefter — omstruktureringen ændrede ikke opførslen, kun overskueligheden.

Fase 13 — Delte knapper og adressesøgning

Skelpunkter og skellinjer blev delt op i to uafhængige knapper, så man kan hente det ene uden nødvendigvis det andet. Derefter tilføjede vi adressesøgning via DAWA (Danmarks Adressers Web API) — en gratis, nøglefri tjeneste, der (i modsætning til matrikeldataene) er bygget til at blive kaldt direkte fra browseren, med indbygget autocomplete mens man skriver.

Fase 14 — Git og GitHub

Gennem hele forløbet blev der committed løbende efter hver fungerende ændring. Projektet blev til sidst forbundet til et GitHub-repository, med et vigtigt sikkerhedstjek undervejs: at bekræfte at .env-filen (med API-nøglen) aldrig blev sporet af Git og dermed aldrig havnede synligt på GitHub.

Opsummering — hvad endte vi med?

Fra et tomt Vite-projekt endte vi med en fungerende, browserbaseret landmålerapp med:

- Interaktivt kort med klik/træk/højreklik-redigering af målepunkter
- Automatisk areal- og omkredsregning
- Fire baggrundskort, inklusiv officielt 10 cm-ortofoto
- Fire koordinatsystemer (WGS84, UTM32N, DKTM3, S34S) med både visning og manuel indtastning
- Rigtige skelpunkter og skellinjer hentet direkte fra Dataforsyningens GraphQL-API
- Adressesøgning med autocomplete
- Et helt dansk brugerinterface
- En ryddelig, opdelt kodebase der er til at arbejde videre i
- Sikker håndtering af API-nøgler via en lokal proxy

Undervejs blev mindst én funktion (matrikelnumre i jordstykker) bevidst fjernet igen, da den underliggende data viste sig upålidelig — et fint eksempel på, at vibe coding ikke kun handler om at tilføje ting, men også om at vide, hvornår man skal trække noget tilbage.

Udarbejdet som logbog over udviklingen af Landmålerberegner-projektet.